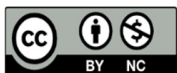


SPRING BOOT TUTORIAL



SPRING BOOT

Em Português (PT-PT)



Luís Simões da Cunha (2025)

Índice

🌀	Parte 1 – Bem-vindo ao Mundo Spring Boot.....	9
🌱	1.1 O que é o Spring Boot e por que é tão popular?	9
📌	Porque é que tantos developers adoram o Spring Boot?	9
🎮	1.2 A magia do autoconfigurador e dos starters	9
📁	O que são os <i>starters</i> ?.....	9
🔧	1.3 Primeira app com Spring Initializr (sem sustos!)	10
	Passo 1 – Vai ao site oficial:.....	10
	Passo 2 – Preenche o formulário assim:.....	10
	Passo 3 – Clica em Generate para fazer download do .zip.	10
	Passo 4 – Abre o projeto com o IntelliJ, Eclipse ou VS Code	10
	Passo 5 – Executa a aplicação.....	10
🏁	Resultado final	11
🚀	Pronto para a próxima?.....	11
🧬	Parte 2 – Estrutura e Anatomia de um Projeto Spring Boot.....	12
📁	2.1 Pastas, ficheiros e o que cada um faz.....	12
📁	Estrutura base de um projeto Maven com Spring Boot.....	12
🌟	Destaques:.....	12
🔧	2.2 Maven vs Gradle: qual usar e porquê?	12
⚙️	Maven	12
⚡	Gradle	13
🎯	2.3 O <code>@SpringBootApplication</code> explicado com clareza.....	13
🧩	Por detrás da cortina.....	13
🔍	Detalhes importantes:	13
🚀	Conclusão da Parte 2	14
🔧	Parte 3 – Propriedades e Configuração como um Pro	15
📁	3.1 <code>application.properties</code> vs <code>application.yml</code>	15
📄	<code>application.properties</code> – O clássico simples.....	15
📄	<code>application.yml</code> – O elegante estruturado	15

👤 3.2 Perfis, variáveis de ambiente e externalização	15
🎯 Perfis (@Profile, spring.profiles.active).....	15
🌐 Variáveis de Ambiente.....	16
📦 Externalizar configs fora do JAR.....	16
🌀 3.3 Boas práticas para organizar configurações.....	16
✅ Nomeia bem e agrupa com @ConfigurationProperties	16
📌 Dicas de ouro:	17
✅ Conclusão da Parte 3	17
🔍 Parte 4 – O Poder do Logging e da Validação	18
📖 4.1 Logging com SLF4J e Log4j2 (sem dor)	18
🛒 Spring Boot usa SLF4J como fachada de logging	18
📦 Para usar Log4j2 , adiciona esta dependência ao pom.xml:.....	18
📁 Cria um ficheiro log4j2.xml em src/main/resources:	18
📖 Usar no código:	18
✅ 4.2 Bean Validation (automática e personalizada)	19
📦 Spring Boot inclui o Bean Validation API (JSR 380) com Hibernate Validator por defeito	19
🎈 Exemplo básico:	19
👤 Como ativar automaticamente:	19
✂️ Validações personalizadas:	20
🎨 4.3 Erros amigáveis para os utilizadores	20
📄 Exemplo de resposta padrão do Spring Boot:	20
👤 Personalizar resposta com @ControllerAdvice:	20
🎯 Conclusão da Parte 4	21
📦 Parte 5 – Persistência com Spring Data JPA.....	22
🔧 5.1 Conectar a base de dados em minutos.....	22
🔧 Passos para configurar:.....	22
📦 5.2 O básico de CrudRepository e JpaRepository	22
👤 Vamos criar a entidade Livro:	22
👤 Agora o repositório mágico:	23
🔮 5.3 Queries mágicas e personalizadas com @Query e Criteria API	23

🔍 Métodos com nomes mágicos:	23
📄 Queries personalizadas com @Query:	23
⚙️ Criteria API – Quando queres flexibilidade programática	23
📌 Resumo da Parte 5	24
🔗 Parte 6 – Relações e Modelos Avançados de Dados	25
✚ 6.1 Um para muitos, muitos para muitos sem dores de cabeça	25
🎯 Objetivo: representar relações reais no modelo de dados.....	25
📄 1:N – Um para muitos (Autor → Livros).....	25
🔄 N:M – Muitos para muitos (Livro ↔ Género)	25
📄 6.2 Paginação e ordenação sem escrever SQL	26
Exemplo com findAll(Pageable pageable):.....	26
👤 6.3 Introdução ao QueryDSL	26
Passos para usar QueryDSL com Spring Boot:.....	27
✅ Conclusão da Parte 6	27
🌐 Parte 7 – REST API com Spring Boot (Parte I).....	28
🕒 7.1 Criar controladores REST com @RestController	28
📌 Exemplo simples:	28
✚ Explicação:	28
🔧 7.2 Testar endpoints com Postman ou curl	28
🛒 Opção 1: Postman (interface gráfica)	28
🛒 Opção 2: curl (linha de comandos).....	28
🛡️ 7.3 Lidar com exceções e respostas padronizadas	29
📌 Exemplo de tratamento global com @RestControllerAdvice:	29
🌟 Resultado:.....	29
✅ Conclusão da Parte 7	30
🔒 Parte 8 – REST API com Spring Boot (Parte II).....	31
🧬 8.1 Versionamento de APIs (URI, header ou param?)	31
1️⃣ Versionamento por URI	31
2️⃣ Versionamento por <i>Header</i> personalizado.....	31
3️⃣ Versionamento por <i>Request Param</i>	31

	8.2 Documentação automática com Swagger/OpenAPI	32
	Adiciona dependências ao pom.xml (Springdoc OpenAPI):.....	32
	Acede à documentação da tua API em:	32
	Personalizar com anotações:	32
	8.3 Segurança com JWT passo a passo	32
	O que é JWT?	32
	Passos práticos:	32
	Conclusão da Parte 8	33
	Parte 9 – Segurança com Spring Security (nível 1).....	34
	9.1 Autenticação básica e formulário.....	34
	Spring Security em ação... automaticamente!.....	34
	Formulário personalizado.....	34
	9.2 Autorização por perfis e papéis	35
	Exemplo com dois utilizadores:	35
	Proteger endpoints por papel:.....	35
	9.3 Injeção de segurança sem drama.....	35
	Quer saber quem está autenticado?.....	35
	Conclusão da Parte 9	36
	Parte 10 – Segurança com Spring Security (nível 2)	37
	10.1 Autenticação com JDBC, LDAP e OAuth2.....	37
	JDBC: Utilizadores numa base de dados relacional	37
	LDAP: Diretório de utilizadores centralizado	37
	OAuth2: Login com Google (ou outro provedor).....	38
	10.2 Remember-me e bloqueio por tentativas falhadas	38
	Remember-me	38
	Bloqueio após tentativas falhadas	39
	10.3 Two-Factor Authentication com Google Authenticator.....	39
	Requisitos:	39
	Etapas:	39
	Conclusão da Parte 10	40
	Parte 11 – Monitorização com Spring Boot Actuator	41

🛒 11.1 Endpoints úteis e como usá-los	41
🔧 No pom.xml:	41
🔒 Por defeito, só o /actuator/health e /actuator/info estão abertos....	41
📡 Alguns endpoints importantes:	41
❤️ 11.2 Criar indicadores de saúde personalizados	41
👨‍⚕️ Exemplo de HealthIndicator personalizado:	41
📊 11.3 Prometheus + Grafana: observabilidade real.....	42
🕒 Passo 1 – Adiciona o Micrometer + Prometheus	42
🔧 application.properties:	42
📡 O endpoint /actuator/prometheus agora mostra métricas no formato Prometheus!	42
💻 Passo 2 – Instalar e ligar ao Grafana	42
✅ Conclusão da Parte 11	43
🌊 Parte 12 – Reactive Programming com WebFlux	44
🧠 O que é Programação Reativa? Vale mesmo a pena?	44
🚧 Criar uma API Reativa com Spring WebFlux	44
🕸️ WebClient, RSocket e WebSocket na prática	45
✅ Resumo Prático	46
👉 Desafio para ti	46
✅ Parte 13 – Testar como um Ninja	47
🧪 13.1 Testes com @SpringBootTest, Mockito e MockMvc	47
🎯 @SpringBootTest	47
👤 @MockBean e Mockito	47
🎯 Testes com MockMvc (REST API).....	47
🧩 13.2 Testes de Integração vs Testes Unitários	48
🧠 Dica cognitiva:.....	48
📊 13.3 Estratégias práticas de cobertura e confiança.....	48
🎯 Dicas práticas:	48
💡 Ferramentas úteis:.....	48
🧠 Exemplo final: cobertura total e teste realista.....	49
✅ Conclusão da Parte 13	49

🧩	Parte 14 – Desdobrar para Produção (Deploy sem Medo).....	50
🎯	Objetivo:.....	50
📦	1. Empacotar como JAR ou WAR – O que escolher?.....	50
☁️	2. Deploy em Heroku – O teu app na nuvem em minutos	50
🐳	3. Docker + Spring Boot = deploy portátil	51
⚙️	4. Kubernetes e OpenShift – Para produção a sério	51
🗣️	5. DevOps com Spring Boot – Dicas valiosas.....	52
✅	Conclusão	52
💡	1. Kotlin com Spring Boot – mais conciso, mais funcional	53
	Porquê Kotlin?	53
	Exemplo básico:.....	53
	Como começar:	53
⚡	2. GraalVM e Native Image – Spring Boot em modo ultraleve	53
	O que é GraalVM?	53
	Vantagens:	54
	Como funciona?	54
	Passos com Maven:.....	54
🌐	3. GraphQL – Alternativa moderna ao REST	54
	O que é GraphQL?.....	54
	Exemplo de query:.....	54
	Como usar com Spring Boot?.....	55
	Também podes usar:.....	55
✅	Conclusão da Parte 15	55

🌟 Parte 1 – Bem-vindo ao Mundo Spring Boot

🌱 1.1 O que é o Spring Boot e por que é tão popular?

Imagina que estás a construir uma casa. O **Spring Framework** é como um conjunto de ferramentas de construção: tens martelos, pregos, tijolos... mas tens de saber usar tudo desde o início. Dá muito poder, mas exige tempo.

Agora entra o **Spring Boot**. Ele é como um **kit de construção automática**. Dás-lhe um plano simples e ele monta grande parte da casa por ti — com canalização, eletricidade e tudo! 💡

🔗 Porque é que tantos developers adoram o Spring Boot?

- 🛠️ **Menos configuração manual** – nada de ficheiros XML infinitos ou servidores externos obrigatórios.
- 🚀 **Apps prontas em minutos** – ideal para protótipos rápidos e microserviços.
- 🧙 **Autoconfiguração mágica** – se usas uma base de dados, ele configura-a por ti!
- 🧱 **Base sólida e extensível** – é compatível com o ecossistema Spring (Security, Data, etc.).
- 📦 **Standalone** – compilas e corres com `java -jar`, sem instalar um servidor como Tomcat à parte.

💡 **Dica Cognitiva:** Começar com uma estrutura pré-configurada reduz o número de decisões iniciais que o cérebro tem de tomar. Isso liberta recursos mentais para aprender o que realmente importa: construir aplicações!

🧙 1.2 A magia do autoconfigurador e dos starters

O **autoconfigurador** é como um assistente inteligente que te prepara o palco consoante os adereços que vê na sala. Se encontra uma base de dados no projeto, ele prepara as ligações. Se encontra o Spring Web, configura um servidor web embutido. ✨

📦 O que são os *starters*?

São **pacotes de dependências prontos a usar**. Em vez de adicionares 7 bibliotecas para fazer uma API REST, adicionas só isto:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

👉 O Spring Boot trata de trazer tudo o que é necessário: Spring MVC, Jackson para JSON, Tomcat embutido... e ainda te dá bom senso por defeito.

💡 *Lembras-te do LEGO?* Os starters são como caixas temáticas com todas as peças certas. Há starter-web, starter-data-jpa, starter-security, etc.

🔧 1.3 Primeira app com Spring Initializr (sem sustos!)

Vamos criar o nosso primeiro projeto como quem encomenda um bolo pronto a decorar!



Passo 1 – Vai ao site oficial:

🔗 <https://start.spring.io>

Passo 2 – Preenche o formulário assim:

Campo	Valor
Project	Maven
Language	Java
Spring Boot	Usa a versão estável mais recente
Group	com.exemplo
Artifact	ola-springboot
Name	OlaSpringBoot
Dependencies	Spring Web

💡 *Dica Cognitiva:* Começar com um único módulo e uma única dependência reduz a carga cognitiva e permite consolidar a compreensão inicial.

Passo 3 – Clica em **Generate** para fazer download do .zip.

Passo 4 – Abre o projeto com o IntelliJ, Eclipse ou VS Code

Se usas VS Code:

- Instala as extensões: Java Extension Pack + Spring Boot Extension Pack.
 - Abre a pasta e aguarda o download automático das dependências via Maven.
-

Passo 5 – Executa a aplicação

No ficheiro `src/main/java/com/exemplo/OlaSpringBootApplication.java` deves ver algo assim:

```
@SpringBootApplication
public class OlaSpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(OlaSpringBootApplication.class, args);
    }
}
```

```
}  
}
```

⚙️ Clica no botão de **Run** ou escreve no terminal:

```
./mvnw spring-boot:run
```

Verás algo como:

```
Tomcat started on port(s): 8080  
Started OlaSpringBootApplication in 2.123 seconds
```

🎉 A tua aplicação está a correr em <http://localhost:8080>

🏁 Resultado final

Já tens uma app Spring Boot **100% funcional**, com servidor embutido e pronta para crescer. Nem foi preciso configurar Tomcat, escrever `web.xml` ou tocar em nenhuma porta obscura da configuração.

🚀 Pronto para a próxima?

Na **Parte 2**, vamos explorar a estrutura do projeto, o `pom.xml`, o poder dos ficheiros `application.properties`, e como tornar o teu projeto verdadeiramente teu!

🔗 Parte 2 – Estrutura e Anatomia de um Projeto Spring Boot

📁 2.1 Pastas, ficheiros e o que cada um faz

Ao gerar um projeto com o Spring Initializr, recibes uma estrutura mínima, clara e organizada — como uma casa acabada de construir, pronta a decorar e personalizar 🏠.

🏠 Estrutura base de um projeto Maven com Spring Boot

```
meu-projeto-springboot
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.exemplo
│   │   │       └── MinhaApp.java ← ponto de entrada (main)
│   │   └── resources
│   │       └── application.properties ← configs da app
│   └── test
│       ├── java
│       │   └── MinhaAppTests.java ← testes
│   └── pom.xml ← o “coração” do Maven
└── mvnw / mvnw.cmd ← executáveis Maven wrapper
```

✦ Destaques:

- pom.xml → gere as dependências, plugins, versões, etc.
- application.properties → configurações da app (porta, BD, segurança...)
- MinhaApp.java (com main) → é onde tudo começa!
- resources/ → onde colocas ficheiros estáticos, templates e configs.

💡 *Dica Cognitiva:* Uma estrutura consistente e enxuta ajuda o cérebro a formar padrões mentais mais rápidos e sólidos — um dos princípios da carga cognitiva germânica.

✂ 2.2 Maven vs Gradle: qual usar e porquê?

Ambos são ferramentas de build. A diferença? ✨ *Pensa no Maven como LEGO tradicional* e o Gradle como LEGO Technic com motores programáveis.

⚙ Maven

- ✦ XML declarativo
- ✦ Comunidade gigantesca

- ◆ Convencional e previsível
- ◆ Mais fácil para iniciantes
- ☒ Usado neste tutorial

```
<!-- Exemplo de dependência no pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

⚡ Gradle

- ◆ DSL em Groovy ou Kotlin
- ◆ Mais rápido, mas com curva de aprendizagem
- ◆ Mais configurável

⚖️ *Resumo:* Para aprender e manter projetos padrão Spring Boot → **Maven** é ideal.

🔗 2.3 O @SpringBootApplication explicado com clareza

Esta anotação é como o “**tudo-em-um**” da tua aplicação Spring Boot. Vê-a como um comando mágico que combina três outras anotações:

```
@SpringBootApplication
public class MinhaApp {
    public static void main(String[] args) {
        SpringApplication.run(MinhaApp.class, args);
    }
}
```

🔗 Por detrás da cortina...

```
@SpringBootApplication =
    @Configuration          // Define beans e configurações
+ @EnableAutoConfiguration // Faz autoconfiguração baseada nas libs
+ @ComponentScan           // Procura componentes (classes anotadas)
```

🔍 Detalhes importantes:

- A `main()` é o ponto de partida, como qualquer app Java.
- `SpringApplication.run(...)` arranca tudo, **incluindo um servidor Tomcat embutido** por defeito!
- Todos os ficheiros `.java` nas subpastas serão analisados por Spring (graças ao `@ComponentScan`).

💡 *Nota pedagógica:* Esta anotação composta ajuda a esconder a complexidade para iniciantes, revelando-a apenas quando necessário — o que é excelente para uma curva de aprendizagem progressiva.

Conclusão da Parte 2

✓ Já compreendes:

- Como está organizada a “casa” do teu projeto Spring Boot
- Por que usamos Maven (e não Gradle neste caso)
- Como o `@SpringBootApplication` facilita a vida e invoca magia ✨

Parte 3 – Propriedades e Configuração como um Pro

3.1 application.properties VS application.yml

application.properties – O clássico simples

- Formato de **chave=valor**
- Ideal para configurações simples e diretas
- Muito usado em tutoriais e projetos pequenos

```
server.port=8081
spring.datasource.url=jdbc:h2:mem:testdb
```

application.yml – O elegante estruturado

- Formato **hierárquico e legível**
- Menos repetição, melhor organização
- Ideal para configurações complexas

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:h2:mem:testdb
```

 *Dica Cognitiva:* A estrutura hierárquica do YAML melhora a memorização visual e a identificação de blocos lógicos de configuração — ideal para quando o projeto cresce!

 Ambos funcionam! Usa .yml quando tens configurações mais complexas ou múltiplos perfis. Usa .properties para algo simples e direto.

3.2 Perfis, variáveis de ambiente e externalização

Perfis (@Profile, spring.profiles.active)

Perfis permitem-te **ter diferentes configurações para diferentes ambientes** (dev, test, prod...).

No application.properties:

```
spring.profiles.active=dev
```

Cria ficheiros específicos:

- application-dev.properties

- application-prod.properties

Cada um pode ter configurações como:

```
server.port=8081 # para dev
```

 **Metáfora:** Perfis são como “roupas” diferentes que a aplicação veste consoante o ambiente. Em casa (dev), podes usar pijama. Em produção, vestes fato completo!

Variáveis de Ambiente

Spring Boot lê automaticamente variáveis do sistema como:

```
export SPRING_DATASOURCE_URL=jdbc:mysql://localhost/db
```

E também permite sobrepor propriedades com parâmetros no arranque:

```
java -jar minhaApp.jar --server.port=9000
```

Externalizar configs fora do JAR

✓ Coloca o ficheiro application.properties na mesma pasta do .jar ✓ Usa --spring.config.location=... para apontar para outro ficheiro:

```
java -jar minhaApp.jar --spring.config.location=file:/caminho/config/
```

3.3 Boas práticas para organizar configurações

Nomeia bem e agrupa com @ConfigurationProperties

Cria uma classe como:

```
@Component
@ConfigurationProperties(prefix = "app")
public class AppConfig {
    private String titulo;
    private String versao;
    // getters e setters
}
```

E configura no .properties:

```
app.titulo=Minha App
app.versao=1.0.1
```

 Isto **desacopla a lógica do código das configurações**, tornando tudo mais limpo e testável.

Dicas de ouro:

-  Agrupa por domínio lógico (db, email, auth)
 -  Nunca coloques passwords diretamente — usa Vault, dotenv ou variáveis de ambiente
 -  Usa ficheiros .env com ferramentas como dotenv-spring
 -  Evita hardcode de configurações dentro do código Java
 -  Remove propriedades antigas ou não usadas — ajudam a manter o projeto limpo e seguro
-

Conclusão da Parte 3

Agora já dominas:

- A diferença entre .properties e .yaml
- Como criar perfis e externalizar configs com segurança
- Boas práticas de organização e separação de responsabilidades

Parte 4 – O Poder do Logging e da Validação

4.1 Logging com SLF4J e Log4j2 (sem dor)

O **logging** (registo de mensagens durante a execução) é como o “diário secreto” da tua aplicação. Ajuda a:

- Entender o que aconteceu
- Diagnosticar erros
- Controlar o comportamento em produção

Spring Boot usa SLF4J como fachada de logging

- Pensa no SLF4J como um “adaptador universal” — liga-se a diversas ferramentas como Logback, Log4j2, etc.
- Por padrão, Spring Boot usa **Logback** como motor de logging.

Para usar **Log4j2**, adiciona esta dependência ao pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-log4j2</artifactId>
</dependency>
```

⚠ Remove o spring-boot-starter-logging (que usa Logback por defeito), senão haverá conflitos!

Cria um ficheiro log4j2.xml em src/main/resources:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss} [%t] %-5level %logger -
%msg%n"/>
    </Console>
  </Appenders>
  <Loggers>
    <Root level="info">
      <AppenderRef ref="Console"/>
    </Root>
  </Loggers>
</Configuration>
```

Usar no código:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger logger =
```

```
LoggerFactory.getLogger(MinhaClasse.class);

logger.info("Mensagem informativa");
logger.warn("Atenção! Algo pode estar errado");
logger.error("Erro crítico: {}", ex.getMessage());
```

 *Dica Cognitiva:* Usar logs claros com níveis (INFO, WARN, ERROR) ajuda a distinguir rapidamente o que é normal e o que é anormal — facilitando a leitura e reduzindo frustração em debugging.

4.2 Bean Validation (automática e personalizada)

Validar os dados de entrada é **crucial** — imagina uma app de banco que aceita criar uma conta com nome = "" 

 Spring Boot inclui o **Bean Validation API (JSR 380)** com **Hibernate Validator** por defeito

 Exemplo básico:

```
public class Utilizador {
    @NotBlank
    private String nome;

    @Email
    private String email;

    @Min(18)
    private int idade;
    // Getters e Setters
}
```

 Como ativar automaticamente:

1. Anota o método com `@Valid`
2. Define um `@RestController`

```
@RestController
public class UtilizadorController {

    @PostMapping("/utilizador")
    public ResponseEntity<String> criar(@Valid @RequestBody Utilizador u) {
        return ResponseEntity.ok("Utilizador criado!");
    }
}
```

Se os dados forem inválidos, Spring responde com 400 Bad Request automaticamente! 

✂ Validações personalizadas:

1. Cria uma **anotação customizada**
2. Cria um **validador com lógica personalizada**

```
@Constraint(validatedBy = NIFValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface NIFValido {
    String message() default "NIF inválido!";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class NIFValidator implements ConstraintValidator<NIFValido,
String> {
    public boolean isValid(String nif, ConstraintValidatorContext ctx) {
        return nif != null && nif.matches("\\d{9}"); // NIF português
    }
}
```

🧠 4.3 Erros amigáveis para os utilizadores

Quando a validação falha, queremos devolver **mensagens claras** — não um stack trace gigante!

📄 Exemplo de resposta padrão do Spring Boot:

```
{
  "timestamp": "2025-04-30T12:34:56",
  "status": 400,
  "errors": [
    "Nome não pode ser vazio",
    "Idade mínima é 18"
  ]
}
```

👤 Personalizar resposta com @ControllerAdvice:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<List<String>>
handleValidationErrors(MethodArgumentNotValidException ex) {
        List<String> erros = ex.getBindingResult()
            .getFieldErrors()
            .stream()
```

```
        .map(e -> e.getDefaultMessage())  
        .toList();  
    return ResponseEntity.badRequest().body(erros);  
}  
}
```

🔍 Isto melhora muito a experiência do utilizador — e a do programador também!

🎯 Conclusão da Parte 4

✓ Já sabes:

- Como configurar logs com SLF4J e Log4j2
- Como validar dados com anotações (@NotBlank, @Email, etc.)
- Como devolver erros claros e compreensíveis aos utilizadores

Parte 5 – Persistência com Spring Data JPA

5.1 Conectar a base de dados em minutos

Spring Boot torna a **ligação a uma base de dados quase automática**. Para começar, vamos usar o **H2**, uma base de dados em memória perfeita para testes.

Passos para configurar:

1. No `pom.xml`, adiciona:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

2. No `application.properties`:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

3. Acede à consola H2 em <http://localhost:8080/h2-console>
(username: sa, sem password)

 *Dica cognitiva:* Começar com H2 reduz a frustração inicial. Não é preciso instalar nada!

5.2 O básico de `CrudRepository` e `JpaRepository`

Vamos criar a entidade `Livro`:

```
@Entity
public class Livro {
  @Id @GeneratedValue
  private Long id;

  private String titulo;
  private String autor;
  private int ano;

  // Getters e setters
}
```

🧙 Agora o repositório mágico:

```
public interface LivroRepository extends JpaRepository<Livro, Long> {  
}
```

🎨 Só com isso, já tens métodos como:

`findAll()`, `findById()`, `save()`, `deleteById()`, `count()`...

🌟 É como se tivesses um **ORM com superpoderes**. Não escreveste SQL e já tens CRUD completo!

🔍 5.3 Queries mágicas e personalizadas com `@Query` e Criteria API

🔍 Métodos com nomes mágicos:

```
List<Livro> findByAutor(String autor);  
List<Livro> findByAnoGreaterThan(int ano);
```

Spring interpreta o nome do método e gera SQL automaticamente! ✨

📄 Queries personalizadas com `@Query`:

```
@Query("SELECT l FROM Livro l WHERE l.ano < :ano")  
List<Livro> livrosAntigos(@Param("ano") int ano);
```

- ✓ Usa JPQL (não SQL puro)
 - ✓ Permite total controlo sem complicação
-

⚙️ Criteria API – Quando queres flexibilidade programática

```
public List<Livro> buscarComFiltro(String autor) {  
    CriteriaBuilder cb = em.getCriteriaBuilder();  
    CriteriaQuery<Livro> cq = cb.createQuery(Livro.class);  
    Root<Livro> root = cq.from(Livro.class);  
    cq.select(root).where(cb.equal(root.get("autor"), autor));  
    return em.createQuery(cq).getResultList();  
}
```

💡 Usa Criteria quando precisas de construir queries dinâmicas em runtime.

Resumo da Parte 5

- ✓ Ligaste a uma base de dados com zero dor
- ✓ Usaste um `JpaRepository` com métodos mágicos
- ✓ Aprendeste a personalizar queries com `@Query` e `Criteria`

Na **Parte 6**, vamos aprofundar as relações entre entidades — um-para-muitos, muitos-para-muitos — e aprender a paginar e ordenar resultados como um “pro”.

Parte 6 – Relações e Modelos Avançados de Dados

6.1 Um para muitos, muitos para muitos sem dores de cabeça

 **Objetivo:** representar relações reais no modelo de dados

Vamos imaginar um sistema de biblioteca:

- Um **autor** pode ter **muitos livros**
- Um **livro** pode ter **muitos géneros**

Estas relações traduzem-se em:

1:N – Um para muitos (Autor → Livros)

```
@Entity
public class Autor {
    @Id @GeneratedValue
    private Long id;

    private String nome;

    @OneToMany(mappedBy = "autor", cascade = CascadeType.ALL)
    private List<Livro> livros = new ArrayList<>();
}

@Entity
public class Livro {
    @Id @GeneratedValue
    private Long id;
    private String titulo;

    @ManyToOne
    private Autor autor;
}
```

 O mappedBy indica que a relação já está mapeada no lado "Livro".

N:M – Muitos para muitos (Livro ↔ Género)

```
@Entity
public class Genero {
    @Id @GeneratedValue
    private Long id;
    private String nome;
```

```

    @ManyToMany(mappedBy = "generos")
    private List<Livro> livros;
}

@Entity
public class Livro {
    @Id @GeneratedValue
    private Long id;
    private String titulo;

    @ManyToMany
    @JoinTable(
        name = "livro_genero",
        joinColumns = @JoinColumn(name = "livro_id"),
        inverseJoinColumns = @JoinColumn(name = "genero_id")
    )
    private List<Genero> generos;
}

```

 *Dica Cognitiva:* Visualizar as relações como ligações reais (um autor e os seus livros, um livro com géneros) ajuda a consolidar o modelo mental dos alunos.

6.2 Paginação e ordenação sem escrever SQL

Quando tens muitos registos, é melhor enviar só um pedaço de cada vez. Para isso usamos **paginação** e **ordenação**, e Spring Boot facilita imenso com Pageable.

Exemplo com findAll(Pageable pageable):

```

@GetMapping("/livros")
public Page<Livro> listar(@RequestParam int page, @RequestParam int size)
{
    Pageable pageable = PageRequest.of(page, size, Sort.by("titulo"));
    return livroRepository.findAll(pageable);
}

```

Resultado: uma página de livros ordenados por título, sem escrever SQL!

 Também funciona com métodos como findByAutor(String autor, Pageable pageable)

6.3 Introdução ao QueryDSL

O QueryDSL é uma ferramenta poderosa para construir queries **tipadas**, **dinâmicas** e com **autocompletar no IDE** — como se estivesses a usar o Java para escrever SQL com segurança.

Passos para usar QueryDSL com Spring Boot:

1. Adiciona ao pom.xml:

```
<dependency>
  <groupId>com.querydsl</groupId>
  <artifactId>querydsl-jpa</artifactId>
</dependency>
<plugin>
  <groupId>com.mysema.maven</groupId>
  <artifactId>apt-maven-plugin</artifactId>
  <version>1.1.3</version>
  <executions>
    <execution>
      <goals>
        <goal>process</goal>
      </goals>
      <configuration>
        <outputDirectory>target/generated-sources/java</outputDirectory>
      </configuration>
    </execution>
  </executions>
</plugin>
<processor>com.querydsl.apt.jpa.JPAAnnotationProcessor</processor>
  </configuration>
</execution>
</executions>
</plugin>
```

2. Após compilar, tens classes QLivro, QAutor geradas automaticamente.
3. Exemplo de uso:

```
QLivro livro = QLivro.livro;
JPAQuery<Livro> query = new JPAQuery<>(entityManager);
List<Livro> resultado = query.select(livro)
    .where(livro.ano.goe(2000))
    .orderBy(livro.titulo.asc())
    .fetch();
```

 Isto é ideal para cenários complexos onde Criteria API seria demasiado verboso.

☒ Conclusão da Parte 6

✓ Já sabes:

- Criar relações 1:N e N:M com elegância
- Pagar e ordenar resultados sem SQL
- Começar com QueryDSL para queries poderosas, seguras e dinâmicas

Parte 7 – REST API com Spring Boot (Parte I)

7.1 Criar controladores REST com @RestController

A anotação `@RestController` transforma uma classe Java comum num **controlador REST**, capaz de receber pedidos HTTP e devolver respostas em formato JSON ou XML. É como o "porteiro" da tua API 🏠.

Exemplo simples:

```
@RestController
@RequestMapping("/api/livros")
public class LivroController {

    @GetMapping
    public List<Livro> listarTodos() {
        return livroRepository.findAll();
    }

    @PostMapping
    public Livro criar(@RequestBody Livro livro) {
        return livroRepository.save(livro);
    }
}
```

Explicação:

- `@RestController`: indica que esta classe vai tratar pedidos REST
- `@RequestMapping`: define o caminho base (/api/livros)
- `@GetMapping`, `@PostMapping`: associam métodos a verbos HTTP

 *Dica Cognitiva:* Usar o nome dos métodos HTTP no nome das anotações (`@GetMapping`, `@PostMapping`) facilita a memorização e evita confusão!

7.2 Testar endpoints com Postman ou curl

Opção 1: Postman (interface gráfica)

1. Abre o Postman e cria um novo pedido GET
`http://localhost:8080/api/livros`
2. Clica em "Send" e vê os dados devolvidos pela tua API 📦

Opção 2: curl (linha de comandos)

```
curl http://localhost:8080/api/livros
```

Para criar um livro:

```
curl -X POST http://localhost:8080/api/livros \
-H "Content-Type: application/json" \
-d '{"titulo":"Dom Quixote", "autor":"Cervantes", "ano":1605}'
```

💡 *Dica:* Usar ferramentas visuais como o Postman ajuda a formar um mapa mental das rotas da API — ótimo para aprender!

🔒 7.3 Lidar com exceções e respostas padronizadas

Não queremos que a app devolva *stack traces* gigantes! Queremos respostas claras e consistentes ✨

📍 Exemplo de tratamento global com `@RestControllerAdvice`:

```
@RestControllerAdvice
public class ErrorHandler {

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<String> naoEncontrado(EntityNotFoundException
ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("Recurso
não encontrado");
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<List<String>>
dadosInvalidos(MethodArgumentNotValidException ex) {
        List<String> erros = ex.getBindingResult().getFieldErrors()
            .stream().map(FieldError::getDefaultMessage).toList();
        return ResponseEntity.badRequest().body(erros);
    }
}
```

✨ Resultado:

```
{
  "status": 400,
  "errors": [
    "Título não pode estar vazio",
    "Ano deve ser maior que 0"
  ]
}
```

💡 *Nota pedagógica:* Isto melhora a experiência de utilizador, evita frustração e dá feedback rápido — fundamental em UX.

Conclusão da Parte 7

✓ Já sabes:

- Criar controladores REST com `@RestController`
- Testar endpoints com Postman ou curl
- Tratar erros e devolver mensagens compreensíveis

Parte 8 – REST API com Spring Boot (Parte II)

8.1 Versionamento de APIs (URI, header ou param?)

Com o tempo, as APIs evoluem. O versionamento permite-te **introduzir mudanças sem quebrar o que já funciona**. Spring Boot permite três estratégias principais:

1 Versionamento por URI

Mais simples e direto:

```
@RequestMapping("/api/v1/livros")
public class LivroControllerV1 { ... }
```

```
@RequestMapping("/api/v2/livros")
public class LivroControllerV2 { ... }
```

✅ Fácil de perceber, ✅ visível nos logs, ❌ pode poluir a estrutura se tiveres muitos endpoints.

2 Versionamento por *Header* personalizado

```
@GetMapping(value = "/livros", headers = "X-API-VERSION=1")
public List<Livro> versao1() { ... }
```

```
@GetMapping(value = "/livros", headers = "X-API-VERSION=2")
public List<Livro> versao2() { ... }
```

✅ Separação elegante, ❌ menos visível, ❌ pode confundir iniciantes

3 Versionamento por *Request Param*

```
@GetMapping(value = "/livros", params = "versao=1")
public List<Livro> versao1() { ... }
```

```
@GetMapping(value = "/livros", params = "versao=2")
public List<Livro> versao2() { ... }
```

✅ Fácil de testar, ✅ evita repetição de URIs, ❌ menos convencional em APIs públicas

 *Dica Cognitiva:* Ensinar as três formas ajuda os alunos a escolher a mais adequada ao seu caso, desenvolvendo pensamento crítico sobre design de APIs.

8.2 Documentação automática com Swagger/OpenAPI

 Adiciona dependências ao pom.xml (Springdoc OpenAPI):

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-ui</artifactId>
  <version>1.7.0</version>
</dependency>
```

 Acede à documentação da tua API em:

<http://localhost:8080/swagger-ui.html>

 Personalizar com anotações:

```
@Operation(summary = "Lista todos os livros", description = "Retorna
todos os livros disponíveis.")
@GetMapping("/livros")
public List<Livro> listar() { ... }
```

💡 A interface do Swagger permite **testar os endpoints interativamente**. Ideal para alunos e testers!

8.3 Segurança com JWT passo a passo

O que é JWT?

Um **token autoassinado** com as credenciais do utilizador (como o e-mail) + tempo de validade. Ideal para autenticação stateless.

Passos práticos:

1 Gerar um token JWT após login:

```
String token = Jwts.builder()
  .setSubject("user@email.com")
  .setIssuedAt(new Date())
  .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 1
  dia
  .signWith(SignatureAlgorithm.HS256, "segredo123")
  .compact();
```

2 Enviar o token no header:

Authorization: Bearer <token>

3 Intercetar e validar com *OncePerRequestFilter*:

```
public class JwtFilter extends OncePerRequestFilter {
  protected void doFilterInternal(HttpServletRequest req,
    HttpServletResponse res, FilterChain chain)
```



```

        throws ServletException, IOException {
    String token = req.getHeader("Authorization");
    if (token != null && token.startsWith("Bearer ")) {
        String jwt = token.substring(7);
        Claims claims =
Jwts.parser().setSigningKey("segredo123").parseClaimsJws(jwt).getBody();
        // Validação feita! Podes configurar autenticação...
    }
    chain.doFilter(req, res);
}
}

```

🔒 Usa uma SecurityConfig personalizada para aplicar este filtro apenas onde necessário (ex: /api/privado/**).

☑ Conclusão da Parte 8

✓ Já sabes:

- Versionar APIs de forma elegante
- Criar documentação interativa com Swagger/OpenAPI
- Aplicar autenticação segura com JWT passo a passo

Parte 9 – Segurança com Spring Security (nível 1)

9.1 Autenticação básica e formulário

Spring Security em ação... automaticamente!

Ao adicionares esta dependência no `pom.xml`, a magia começa:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Ao iniciares a app, vais reparar que:

- A aplicação exige login.
- Um utilizador `user` é criado por defeito.
- A consola mostra uma **password gerada aleatoriamente**.

 Acede a `http://localhost:8080` e usa as credenciais:

Username: user
Password: (ver consola)

Formulário personalizado

Queres uma página de login personalizada? Cria um HTML simples e depois configura a segurança:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/login", "/css/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .formLogin()
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard", true)
                .permitAll();
    }
}
```

 *Dica cognitiva:* Começar com `formLogin()` permite perceber rapidamente o fluxo de autenticação do utilizador.

9.2 Autorização por perfis e papéis

Queres proteger certas partes da app com base no papel do utilizador? Vamos definir perfis como "ADMIN" ou "USER".

Exemplo com dois utilizadores:

```
@Override
protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
    auth.inMemoryAuthentication()
        .withUser("pedro").password("{noop}1234").roles("USER")
        .and()
        .withUser("ana").password("{noop}admin").roles("ADMIN");
}
```

{noop} diz ao Spring Security para **não codificar a password** (apenas para testes!).

Proteger endpoints por papel:

```
http
    .authorizeRequests()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .antMatchers("/user/**").hasAnyRole("USER", "ADMIN")
        .anyRequest().authenticated();
```

✓ /admin/** → só acessível por ADMIN

✓ /user/** → acessível por USER ou ADMIN

9.3 Injeção de segurança sem drama

Quer saber quem está autenticado?

```
@GetMapping("/me")
public String infoUtilizador(@AuthenticationPrincipal User user) {
    return "Olá, " + user.getUsername();
}
```

Ou mais detalhado:

```
Authentication auth =
SecurityContextHolder.getContext().getAuthentication();
String username = auth.getName();
```

 Ideal para **auditoria**, **logging** ou mostrar dados personalizados ao utilizador.

☒ Conclusão da Parte 9

- ✓ Configuraste autenticação com formulário ou básica
 - ✓ Definiste papéis e restringiste acesso a rotas
 - ✓ Obtiveste info do utilizador autenticado com facilidade
-

Parte 10 – Segurança com Spring Security (nível 2)

10.1 Autenticação com JDBC, LDAP e OAuth2

JDBC: Utilizadores numa base de dados relacional

1. Cria uma tabela com o nome `users` e `authorities` (Spring espera esta estrutura):

```
CREATE TABLE users (  
  username VARCHAR(50) NOT NULL PRIMARY KEY,  
  password VARCHAR(100) NOT NULL,  
  enabled BOOLEAN NOT NULL  
);
```

```
CREATE TABLE authorities (  
  username VARCHAR(50) NOT NULL,  
  authority VARCHAR(50) NOT NULL,  
  CONSTRAINT fk_user FOREIGN KEY(username) REFERENCES users(username)  
);
```

2. Configura o `application.properties`:

```
spring.datasource.url=jdbc:h2:mem:testdb  
spring.datasource.driver-class-name=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=
```

3. Define a segurança com `jdbcAuthentication()`:

```
@Override  
protected void configure(AuthenticationManagerBuilder auth) throws  
Exception {  
    auth.jdbcAuthentication().dataSource(dataSource);  
}
```

LDAP: Diretório de utilizadores centralizado

Spring Boot suporta LDAP out-of-the-box. No `application.properties`:

```
spring.ldap.urls=ldap://localhost:8389/  
spring.ldap.base=dc=exemplo,dc=com  
spring.ldap.username=cn=admin,dc=exemplo,dc=com  
spring.ldap.password=admin
```

Na configuração:

```
@Override
protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
    auth.ldapAuthentication()
        .userDnPatterns("uid={0},ou=users")
        .contextSource()
        .url("ldap://localhost:8389/dc=exemplo,dc=com");
}
```

Ideal para empresas com diretórios centralizados! 🏢

🌐 OAuth2: Login com Google (ou outro provedor)

Adiciona ao pom.xml:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Configura o application.yml:

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: SEU_CLIENT_ID
            client-secret: SEU_CLIENT_SECRET
            scope: profile, email
```

Accede a /oauth2/authorization/google para iniciar sessão com Google! ☁️🔑

🔒 10.2 Remember-me e bloqueio por tentativas falhadas

🔒 Remember-me

No configure(HttpSecurity http):

```
http
    .rememberMe()
    .key("segredoSeguro")
    .tokenValiditySeconds(7 * 24 * 60 * 60); // 7 dias
```

🔑 Permite que o utilizador continue autenticado mesmo após fechar o browser.

🔒 Bloqueio após tentativas falhadas

Cria uma lógica simples com contadores em memória ou BD:

```
if (falhas >= 5) {  
    // Bloquear login  
}
```

Ou usa um `AuthenticationFailureHandler` personalizado para contar e reagir:

```
public class CustomAuthFailureHandler implements  
AuthenticationFailureHandler {  
    @Override  
    public void onAuthenticationFailure(...) {  
        // Atualiza contador de falhas  
        // Redireciona com mensagem de erro  
    }  
}
```

👮 Ideal para proteger contra *brute force attacks*.

📱 10.3 Two-Factor Authentication com Google Authenticator

Requisitos:

- Biblioteca como [Google Authenticator Java](#)
- Código QR gerado para o utilizador

Etapas:

1. Gera o segredo:

```
GoogleAuthenticator gAuth = new GoogleAuthenticator();  
GoogleAuthenticatorKey key = gAuth.createCredentials();  
String secret = key.getKey();
```

2. Mostra o QR code (por ex., com Google Charts API):

```
https://chart.googleapis.com/chart?cht=qr&chs=200x200&chl=otpauth://totp/  
{username}?secret={secret}
```

3. Valida o código introduzido:

```
boolean isCodeValid = gAuth.authorize(secret, codigoIntroduzido);
```

Se válido, permite login. Caso contrário, bloqueia.

🔒 Com isto, mesmo que alguém descubra a password, não consegue aceder sem o código gerado no telemóvel!

Conclusão da Parte 10

- ✓ Autenticação profissional com **JDBC**, **LDAP** ou **OAuth2**
- ✓ Funcionalidades avançadas como **remember-me** e **bloqueio por tentativas**
- ✓ **Two-Factor Authentication** para máxima segurança

Parte 11 – Monitorização com Spring Boot Actuator

11.1 Endpoints úteis e como usá-los

O **Spring Boot Actuator** expõe uma série de endpoints REST para **monitorizar, auditar e gerir** a aplicação. Primeiro, instala o módulo:

 No pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

 Por defeito, só o /actuator/health e /actuator/info estão abertos.

Para expor todos os endpoints:

```
management.endpoints.web.exposure.include=*
```

 Alguns endpoints importantes:

Endpoint	Função
/actuator/health	Verifica o estado da aplicação
/actuator/info	Dados informativos (versão, build)
/actuator/metrics	Métricas como memória, CPU, GC
/actuator/env	Variáveis de ambiente e propriedades
/actuator/beans	Lista todos os beans carregados

 *Dica pedagógica:* Estes endpoints são extremamente úteis em produção e podem ser integrados facilmente em sistemas de monitorização.

11.2 Criar indicadores de saúde personalizados

O endpoint /actuator/health pode ser estendido para verificar o estado **de serviços externos** como bases de dados, APIs, filas, etc.

 Exemplo de HealthIndicator personalizado:

```
@Component
public class MeuServicoHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
```

```

        boolean servicoOnline = verificaServicoExterno(); // lógica tua
        if (servicoOnline) {
            return Health.up().withDetail("serviço", "ativo").build();
        } else {
            return Health.down().withDetail("serviço", "inativo").build();
        }
    }
}

```

✅ O estado será incluído no JSON de /actuator/health

```

{
  "status": "UP",
  "components": {
    "meuServico": {
      "status": "DOWN",
      "details": {
        "serviço": "inativo"
      }
    }
  }
}

```



11.3 Prometheus + Grafana: observabilidade real



Passo 1 – Adiciona o Micrometer + Prometheus

```

<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

```



`application.properties:`

```

management.endpoints.web.exposure.include=*
management.endpoint.prometheus.enabled=true
management.metrics.export.prometheus.enabled=true

```



O endpoint /actuator/prometheus agora mostra métricas no formato Prometheus!



Passo 2 – Instalar e ligar ao Grafana

1. Instala o Prometheus e Grafana (podes usar Docker)
2. Configura o `prometheus.yml`:

```

scrape_configs:
  - job_name: 'spring-boot-app'

```

```
static_configs:  
  - targets: ['localhost:8080']
```

3. No Grafana:

- Cria um *Data Source* do tipo Prometheus
- Importa o dashboard ID 4701 da comunidade para Spring Boot

🎯 Agora tens métricas visuais em tempo real: CPU, memória, uptime, latência de endpoints, etc.

☑ Conclusão da Parte 11

- ✓ Usaste o **Spring Boot Actuator** para obter diagnósticos em tempo real
- ✓ Criaste **indicadores de saúde personalizados**
- ✓ Ligaste a aplicação ao **Prometheus e Grafana** para visualização profissional

Parte 12 – Reactive Programming com WebFlux

“Programar de forma reativa é como surfar uma onda de eventos... sem afogar a tua aplicação!”

O que é Programação Reativa? Vale mesmo a pena?

Imagina uma aplicação que lida com milhares de pedidos por segundo, como um site de streaming ou uma app de bolsa. Se usares o modelo tradicional (bloqueante), cada pedido espera pela resposta. Mas com **programação reativa**, podes tratar muitos pedidos em simultâneo, sem bloquear **threads** – como um restaurante que serve vários clientes com uma só fila fluida de pratos!

✦ Conceitos-chave:

- **Backpressure**: evita que um serviço sobrecarregado rebente.
- **Reactor**: biblioteca usada pelo Spring WebFlux para lidar com `Mono<T>` (1 valor) e `Flux<T>` (vários valores).
- **Non-blocking I/O**: liberta a CPU enquanto espera pelos dados.

Quando vale a pena?

- Quando queres escalar muito com poucos recursos.
 - Em microserviços com chamadas encadeadas.
 - Em apps de tempo real (ex: chat, trading).
-

Criar uma API Reativa com Spring WebFlux

Spring WebFlux usa `@RestController` e `RouterFunction` para expor APIs. Aqui está o esqueleto básico:

Dependência Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Controlador com Mono e Flux

```
@RestController
public class ProdutoController {

    @GetMapping("/produto/{id}")
    public Mono<Produto> getProduto(@PathVariable String id) {
        return produtoService.findById(id);
    }
}
```

```

    @GetMapping("/produtos")
    public Flux<Produto> getTodosProdutos() {
        return produtoService.findAll();
    }
}

```

👉 A diferença é que usas Mono para **um** resultado e Flux para **vários**.

🌐 WebClient, RSocket e WebSocket na prática

🌐 *WebClient: o substituto do RestTemplate*

```

WebClient client = WebClient.create("http://localhost:8080");
Mono<Produto> produto = client.get()
    .uri("/produto/123")
    .retrieve()
    .bodyToMono(Produto.class);

```

Podes configurá-lo com headers, timeouts, filtros de log, etc. Totalmente assíncrono!

📡 *RSocket: comunicação reativa bidirecional*

💬 Quatro modos de interação:

- Fire-and-forget 🔥
- Request-response 📩
- Request-stream 🌊
- Channel ↔

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>

```

Permite criar APIs ainda mais rápidas e resilientes, ideais para microserviços avançados.

💬 *WebSocket com STOMP*

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>

```

Perfeito para apps de chat, notificações em tempo real ou dashboards de stock. Usa tópicos e mensagens JSON para comunicação dinâmica entre clientes e servidor.

☑ Resumo Prático

Ferramenta	Uso	Ideal para
WebFlux	APIs reativas com HTTP	Escalabilidade, performance
WebClient	Chamadas assíncronas a APIs	Substituto do RestTemplate
RSocket	Comunicação bidirecional leve	Microserviços, eventos
WebSocket	Mensagens em tempo real	Chats, jogos, dashboards

🔗 Desafio para ti

Cria uma mini API de tarefas reativa com `WebFlux`, um cliente com `WebClient` e tenta ligar um chat com `WebSocket`. Assim, dominas os três pilares da comunicação reativa com Spring Boot!

☑ Parte 13 – Testar como um Ninja

13.1 Testes com @SpringBootTest, Mockito e MockMvc

@SpringBootTest

Esta anotação carrega o **contexto completo da aplicação**, ideal para testes de integração.

```
@SpringBootTest
class ProdutoServiceTest {
    @Autowired
    ProdutoService produtoService;

    @Test
    void testaProdutoExiste() {
        assertTrue(produtoService.existe("prod123"));
    }
}
```

@MockBean e Mockito

Permite simular (mock) comportamentos de beans para testes unitários mais rápidos.

```
@SpringBootTest
class ProdutoServiceTest {

    @MockBean
    ProdutoRepository produtoRepository;

    @Autowired
    ProdutoService produtoService;

    @Test
    void testaProdutoMockado() {
        when(produtoRepository.existsById("123")).thenReturn(true);
        assertTrue(produtoService.existe("123"));
    }
}
```

 Usa Mockito.when(...).thenReturn(...) para controlar o comportamento de métodos externos ao que queres testar.

Testes com MockMvc (REST API)

Ideal para testar controladores sem subir o servidor inteiro!

```

@WebMvcTest(ProdutoController.class)
class ProdutoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    ProdutoService produtoService;

    @Test
    void deveRetornarProdutoComSucesso() throws Exception {
        Produto p = new Produto("123", "Café");
        when(produtoService.getProduto("123")).thenReturn(p);

        mockMvc.perform(get("/produtos/123"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nome").value("Café"));
    }
}

```

13.2 Testes de Integração vs Testes Unitários

Tipo de Teste	O que testa	Exemplo típico
Teste Unitário	Uma classe isolada	Um serviço com dependências mock
Teste de Integração	Múltiplos componentes reais	Um controller com base de dados

Dica cognitiva:

- Usa **unitários** para **lógica de negócio**.
 - Usa **integração** para **verificar tudo a funcionar junto**.
-

13.3 Estratégias práticas de cobertura e confiança

Dicas práticas:

- ✓ Foca-te primeiro nos serviços com mais regras de negócio
- ✓ Usa mocks apenas quando necessário — evita sobreuso
- ✓ Escreve testes para os casos esperados *e* para os inesperados!
- ✓ Usa `@DataJpaTest` para testar apenas a camada de repositórios
- ✓ Automatiza com CI (GitHub Actions, GitLab, Jenkins...)

Ferramentas úteis:

- JaCoCo → Gera relatórios de cobertura

- Testcontainers → Para testes com BD real (ex: PostgreSQL)
 - AssertJ ou Hamcrest → Assertivas mais expressivas
-

Exemplo final: cobertura total e teste realista

```
@SpringBootTest
@AutoConfigureMockMvc
class APICompletaTest {

    @Autowired
    MockMvc mockMvc;

    @Test
    void testaFluxoCompleto() throws Exception {
        mockMvc.perform(post("/clientes")
            .contentType("application/json")
            .content("{\"nome\": \"Joana\"}"))
            .andExpect(status().isCreated());

        mockMvc.perform(get("/clientes"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].nome").value("Joana"));
    }
}
```

Conclusão da Parte 13

- ✓ Dominas @SpringBootTest, Mockito, MockMvc
- ✓ Sabes a diferença entre testes unitários e de integração
- ✓ Conheces boas práticas de cobertura e confiança

🔗 Parte 14 – Desdobrar para Produção (Deploy sem Medo)

🎯 Objetivo:

Aprender a empacotar, desdobrar (deploy) e manter uma aplicação Spring Boot em produção com confiança — seja com JARs, WARs, Docker ou até Kubernetes e OpenShift!

📦 1. Empacotar como JAR ou WAR – O que escolher?

🔗 O básico:

Spring Boot permite-te criar **executáveis standalone**:

- JAR: formato mais comum e recomendado com Spring Boot.
- WAR: só se precisares mesmo de integrar com um servidor de aplicações como Tomcat ou WebLogic.

☁️ *Maven para criar um JAR:*

```
mvn clean package
```

Cria um target/minha-app.jar que podes correr com:

```
java -jar target/minha-app.jar
```

💡 *Dica de pro:*

Para WAR, muda no pom.xml:

```
<packaging>war</packaging>
```

E usa uma classe `SpringBootServletInitializer` para integração com o container externo.

☁️ 2. Deploy em Heroku – O teu app na nuvem em minutos

🌟 *Porquê o Heroku?*

- Fácil, rápido, gratuito para testes.
- Usa Git para deploys simples.

🚀 *Passos:*

1. Cria conta: heroku.com
2. Instala CLI:

```
https://devcenter.heroku.com/articles/heroku-cli
```

3. Login:

```
heroku login
```

4. Cria app:

```
heroku create minha-app-spring
```

5. Faz push:

```
git push heroku main
```

6. Abre no browser:

```
heroku open
```

Simples, mágico e em produção! 🌍

3. Docker + Spring Boot = deploy portátil

Cria a imagem:

```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=meuapp:v1
```

Corre localmente:

```
docker run -p 8080:8080 meuapp:v1
```

Boas práticas:

- Usa `.dockerignore` para não incluir lixo.
- Define variáveis de ambiente via `-e`.

4. Kubernetes e OpenShift – Para produção a sério

Kubernetes:

- Usa `kubectl`, `Kind` ou `Minikube` para testes locais.
- Define `Deployment`, `Service`, `ConfigMap`.

Exemplo de `Deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: minha-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: minha-app
  template:
```

```
metadata:
  labels:
    app: minha-app
spec:
  containers:
  - name: minha-app
    image: meuapp:v1
    ports:
    - containerPort: 8080
```

OpenShift:

- Plataforma empresarial com mais ferramentas.
 - Cria conta em: cloud.redhat.com
 - Usa o **Developer Console** para importar de Git diretamente e gerar pods, services, etc. em modo visual!
-

5. DevOps com Spring Boot – Dicas valiosas

CI/CD:

- Integra com GitHub Actions, GitLab CI ou Jenkins.
- Exemplo simples de `.github/workflows/build.yml`:

```
name: Build
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v2
    - name: Build
      run: mvn clean package
```

Monitoriza:

- Usa Actuator + Prometheus + Grafana (visto na Parte 11!)
- Logging centralizado com ELK Stack ou Grafana Loki

Segurança:

- Evita deixar passwords hardcoded – usa config externa!
 - Encripta com `spring-boot-starter-config` + Vault
-

Conclusão

Com Spring Boot, podes ir da ideia ao deploy em questão de minutos. Não importa se escolhes Heroku, Docker, Kubernetes ou OpenShift — o importante é conheceres as tuas ferramentas e maneres o processo automatizado e seguro 🗝️

Parte 15 – Explorando o Futuro do Spring Boot

Este capítulo que abre portas para o que está a transformar o ecossistema Spring: **Kotlin**, **GraalVM** e **GraphQL**. Vamos tornar isto acessível, prático e motivador, mesmo que estes conceitos pareçam “nível Jedi” no início 🌟

1. Kotlin com Spring Boot – mais conciso, mais funcional

Porquê Kotlin?

Kotlin é uma linguagem moderna e concisa da JetBrains (criadores do IntelliJ) que:

- Elimina muito do “boilerplate” do Java
- Suporta programação funcional e null-safety
- Integra-se perfeitamente com Spring Boot

Exemplo básico:

```
@RestController
class HelloController {
    @GetMapping("/hello")
    fun hello(): String = "Olá, Kotlin com Spring Boot!"
}
```

Como começar:

1. No **Spring Initializr**, escolhe Kotlin como linguagem
2. Usa a extensão `.kt` e estrutura habitual de pacotes
3. Usa `build.gradle.kts` ou `pom.xml` com as dependências Kotlin

Spring Boot 3 oferece suporte oficial ao Kotlin com DSLs específicas para **Spring Security** e **Ktor-style routers**. É um ótimo ponto de partida para novas apps que valorizam produtividade e clareza de código.

2. GraalVM e Native Image – Spring Boot em modo ultraleve

O que é GraalVM?

Uma **máquina virtual avançada** que permite compilar aplicações Java (incluindo Spring Boot) para **executáveis nativos**: binários que arrancam em milissegundos e usam muito menos memória.

Vantagens:

- ✓ Arranque quase instantâneo
- ✓ Ideal para microserviços e containers
- ✓ Imagem única, sem dependência de JVM externa

Como funciona?

- Usa o **Spring AOT plugin** para preparar o código
- Compila para binário nativo com **GraaVM Native Image**

Passos com Maven:

```
mvn spring-boot:build-image -Pnative
```

Ou usa o plugin direto:

```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
</plugin>
```

💡 Usa `spring-native` e `buildpacks` para facilitar o processo. Ideal em conjunto com Docker e CI/CD.

🌐 3. GraphQL – Alternativa moderna ao REST

REST é poderoso, mas pode ser ineficiente quando o cliente precisa de múltiplas chamadas para obter dados dispersos. GraphQL resolve isso!

O que é GraphQL?

Uma **linguagem de query para APIs** que permite ao cliente:

- Pedir **só os dados que quer**
- Evitar `underfetching/overfetching`
- Obter múltiplos recursos numa única chamada

Exemplo de query:

```
query {
  produto(id: "123") {
    nome
    preco
    categoria {
      nome
    }
  }
}
```

Como usar com Spring Boot?

1. Adiciona dependência:

```
<dependency>
  <groupId>com.graphql-java-kickstart</groupId>
  <artifactId>graphql-spring-boot-starter</artifactId>
</dependency>
```

2. Define o schema .graphqls

```
type Produto {
  id: ID!
  nome: String!
  preco: Float!
}
type Query {
  produto(id: ID!): Produto
}
```

3. Implementa o resolver em Java ou Kotlin:

```
@Component
public class ProdutoResolver implements GraphQLQueryResolver {
  public Produto produto(String id) {
    return repo.findById(id);
  }
}
```

Também podes usar:

- **Spring for GraphQL** (suporte oficial)
- **GraphQL over WebSocket** para atualizações em tempo real

Conclusão da Parte 15

 Exploraste três caminhos promissores:

- **Kotlin** para código mais conciso e moderno
- **GraalVM** para desempenho e footprint ultra reduzido
- **GraphQL** para APIs flexíveis e ricas